

Lab 3 – Architectural Justification

Problem Statement #26: Peer-to-Peer Carpooling & Expense Splitter

Kanchhit kumar | SRN: PES1UG24CS213 | Section D | PES University

Architecture Selection

“We chose a Microservices Architecture for the Peer-to-Peer Carpooling & Expense Splitter System.”

The Rider/Driver Mobile App talks only to an **API Gateway**, which routes requests to five independently deployable services: **User & Route** (FR-002), **Ride Matching** (FR-001, FR-003, UC-03/UC-04), **Cost Split** (FR-004), **Payment** and **Rating** (FR-005). Each service owns its data store and exposes a defined interface. Ride Matching orchestrates a booking by calling **ICostSplit** and **IPayment**, and uses an external Maps/Routing API for detour distances.

Reason 1 – Uneven, peak-hour load on route matching

Route compatibility checking (every active driver route tested against the 1.5 km detour tolerance) is geospatially expensive and spikes during morning and evening commute windows, while registration, cost splitting and rating remain light. With microservices we can scale only the Ride Matching service horizontally to meet NFR-002 (95th-percentile match results under 3 s at 500 concurrent users) instead of replicating the entire application.

Reason 2 – Fault isolation around the external payment gateway

Payment depends on a third-party gateway, and UC-03 already expects failures (alternate flow 9a: retry, then cancel after two attempts). Because Payment is a separate service with its own timeouts and retries, a slow or failing gateway cannot block riders from searching or being matched. The services also line up with our Jira epics (Route Matching, Fuel Cost & Payments, Trust & Safety), so each can be changed and redeployed without touching the others.

Security Advantage – Sensitive data isolated with least privilege

Driving-licence documents, travel histories and ratings (NFR-001) live only in the User & Route and Rating services' own encrypted databases (AES-256 at rest). Other services receive only what they need; Ride Matching gets route geometry and user IDs, never licence documents. The API Gateway is the single public entry point: it terminates TLS, validates JWT tokens and rate-limits clients, while internal services are not reachable from the internet. Only the Payment Service holds gateway credentials and stores tokenised references, so a compromise of one service does not expose all user data.

Performance Benefit – Independent scaling of the latency-critical path

Only the search-and-match path is latency-critical. Ride Matching instances can be added during commute peaks and keep active driver routes cached in memory, so the detour check does not hit the database for every request. The Cost Split calculation is a lightweight, stateless call. In a client-server or monolithic design, one server would handle matching, payments and ratings together, and peak-hour matching load would slow every other feature.

Alternatives Considered and Limitations

Layered (monolith) is simpler to build and test and would suit a small pilot, but it must be scaled as a whole and a hung payment call can affect all features. **Client-Server** offers centralised control and easy data consistency, but a single server is a scalability bottleneck at commute peaks and a single point of failure. **Limitations of our choice:** higher operational complexity (deployment, monitoring and service discovery for several services), extra network hops between services, and cross-service consistency. A booking spans Ride Matching and Payment, so it needs a compensating step (cancel the ride and notify both users if payment fails twice), exactly as UC-03 flow 9a describes.